ASSIGNMENT 2 – CO2

TASK 05

Topic: Generic Leaderboard Using Comparator and Sorting

Course: CSA0903 – Programming in Java

Problem Context: Railway e-Ticket Generation

Name: Sowmiya. D

Register Number: 192511063

Aim

- To implement a generic Leaderboard using Java Generics and Comparator to rank participants based on scores and additional tie-breaking conditions.

1.1 Generic Leaderboard

- A generic class allows the same class to work with different data types without writing separate classes for each type.
- In this task, the Leaderboard stores participants and sorts them according to the selected comparison rules.

1.2 Comparator

- Comparator is used to define custom sorting rules.
- The leaderboard uses the following ranking conditions:
- Higher score receives a higher rank.
- If scores are equal, fewer completion times receive a higher rank.
- If both score and completion time are equal, participant name is compared alphabetically.

1.3 Comparator Chaining

- Comparator chaining combines multiple comparison conditions using methods such as:
- comparingInt()
- thenComparingInt()
- thenComparing()
- reversed()
- The comparator first checks the score, then completion time, and finally the participant name.

1.4 Test Cases

Test	Condition	Expected result
Test 1	Different scores	Higher score ranks first
Test 2	Equal scores	Lower completion time ranks first
Test 3	Equal scores and times	Alphabetical name order

1.5 Time Complexity

Operation	Complexity
Add participant	O(1) average
Compare two participants	O(1)
Sort n participants	O(n log n)
Display leaderboard	O(n)

1.6 Alternative – Comparable and Stable Sorting

Comparable defines the natural ordering inside the participant class.

Comparator allows multiple external sorting rules.

Java's List.sort() uses a stable sorting algorithm, meaning equal elements retain their previous relative order when the comparator considers them equal.

Comparator is more flexible when the application needs different leaderboard ranking methods.

1.7 Conclusion

The generic Leaderboard provides reusable participant management. Comparator chaining supports multiple ranking conditions and ensures that participants are sorted consistently. It is useful in gaming competitions, sports scoring systems and online ranking applications.

2. Java program – Main.java

Create Main.java inside Assignments 02\Task05.

The program includes:

A generic Leaderboard<T> class.

Comparator chaining with three ranking conditions.

Completed TODO 1 and TODO 2.

Three test cases to verify ranking and tie-breaking.

Coding:

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

// Represents a participant
class Player {

private String name;
private int score;
private int completionTime;

public Player(String name, int score, int completionTime) {
this.name = name;
this.score = score;
this.completionTime = completionTime;
}

public String getName() {
return name;
}

public int getScore() {
return score;
}

public int getCompletionTime() {
```

```
        return completionTime;
    }

    @Override
    public String toString() {
        return name + " | Score: " + score
                + " | Time: " + completionTime + " seconds";
    }
}

// Generic Leaderboard class
class Leaderboard<T> {

    private List<T> participants = new ArrayList<>();
    private Comparator<T> comparator;

    public Leaderboard(Comparator<T> comparator) {
        this.comparator = comparator;
    }

    // Add a participant
    public void addParticipant(T participant) {
        participants.add(participant);
    }

    // TODO 1 completed: Sort participants using Comparator
    public List<T> getRankings() {
```

```
        List<T> sortedList = new ArrayList<>(participants);

        // TODO 2 completed: Apply comparator sorting
        sortedList.sort(comparator);

        return sortedList;
    }
}

public class Main {

    public static void main(String[] args) {

        // Comparator chaining:
        // 1. Higher score first
        // 2. Lower completion time first
        // 3. Alphabetical name order
        Comparator<Player> playerComparator =
                Comparator.comparingInt(Player::getScore)
                        .reversed()
                        .thenComparingInt(Player::getCompletionTime)
                        .thenComparing(
                                Player::getName,
                                String.CASE_INSENSITIVE_ORDER
                        );

        Leaderboard<Player> leaderboard =
```

```
        new Leaderboard<>(playerComparator);

// Add participants
leaderboard.addParticipant(new Player("Kumar", 90, 120));
leaderboard.addParticipant(new Player("Arun", 95, 150));
leaderboard.addParticipant(new Player("Divya", 90, 100));
leaderboard.addParticipant(new Player("Bala", 90, 100));
leaderboard.addParticipant(new Player("Priya", 95, 130));

// Display final leaderboard
System.out.println("===== PLAYER LEADERBOARD =====");

List<Player> rankings = leaderboard.getRankings();

for (int i = 0; i < rankings.size(); i++) {
    System.out.println(
            (i + 1) + ". " + rankings.get(i)
    );
}

// Test 1: Higher score comes first
System.out.println("\nTEST 1: Score Priority");
System.out.println(
        rankings.get(0).getName()
        + " ranks first because of the highest score."
);
```

```
        // Test 2: Lower completion time breaks score ties
        System.out.println("\nTEST 2: Completion Time Tie-break");
        System.out.println(
                "Among 90-score players, "
                + rankings.get(2).getName()
                + " ranks before "
                + rankings.get(3).getName()
                + " because of lower completion time."
        );

        // Test 3: Alphabetical order breaks remaining ties
        System.out.println("\nTEST 3: Alphabetical Tie-break");
        System.out.println(
                "Among players with equal score and time, "
                + rankings.get(2).getName()
                + " ranks before "
                + rankings.get(3).getName()
                + " alphabetically."
        );

        System.out.println("\nAll leaderboard tests completed.");
    }
}
```

Output:

```
===== PLAYER LEADERBOARD =====
```

1. Priya | Score: 95 | Time: 130 seconds

2. Arun | Score: 95 | Time: 150 seconds

3. Bala | Score: 90 | Time: 100 seconds

4. Divya | Score: 90 | Time: 100 seconds

5. Kumar | Score: 90 | Time: 120 seconds

TEST 1: Score Priority

Priya ranks first because of the highest score.

TEST 2: Completion Time Tie-break

Bala ranks before Divya because of lower completion time.

TEST 3: Alphabetical Tie-break

Bala ranks before Divya alphabetically.

All leaderboard tests completed.

Implementation:

